In [ ]:

```
                                                    Practical No : 04

Name : Dhanke Anuj Mahadev
Roll No : 20
Batch : A1
Problem Statement : ImplementK-NearestNeighborsalgorithmondiabetes.csvdataset.Compute co
matrix, accuracy, error rate, precision and recall on the given dataset.
```

In [1]:

```
import pandas as pd
import seaborn as sns
```

In [2]:

```
df = pd.read_csv("diabetes.csv")
```

In [3]:

```
df
```

Out[3]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Pedigree	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1
...	...	...	...	...	...	...	...	...	...
763	10	101	76	48	180	32.9	0.171	63	0
764	2	122	70	27	0	36.8	0.340	27	0
765	5	121	72	23	112	26.2	0.245	30	0
766	1	126	60	0	0	30.1	0.349	47	1
767	1	93	70	31	0	30.4	0.315	23	0

768 rows × 9 columns

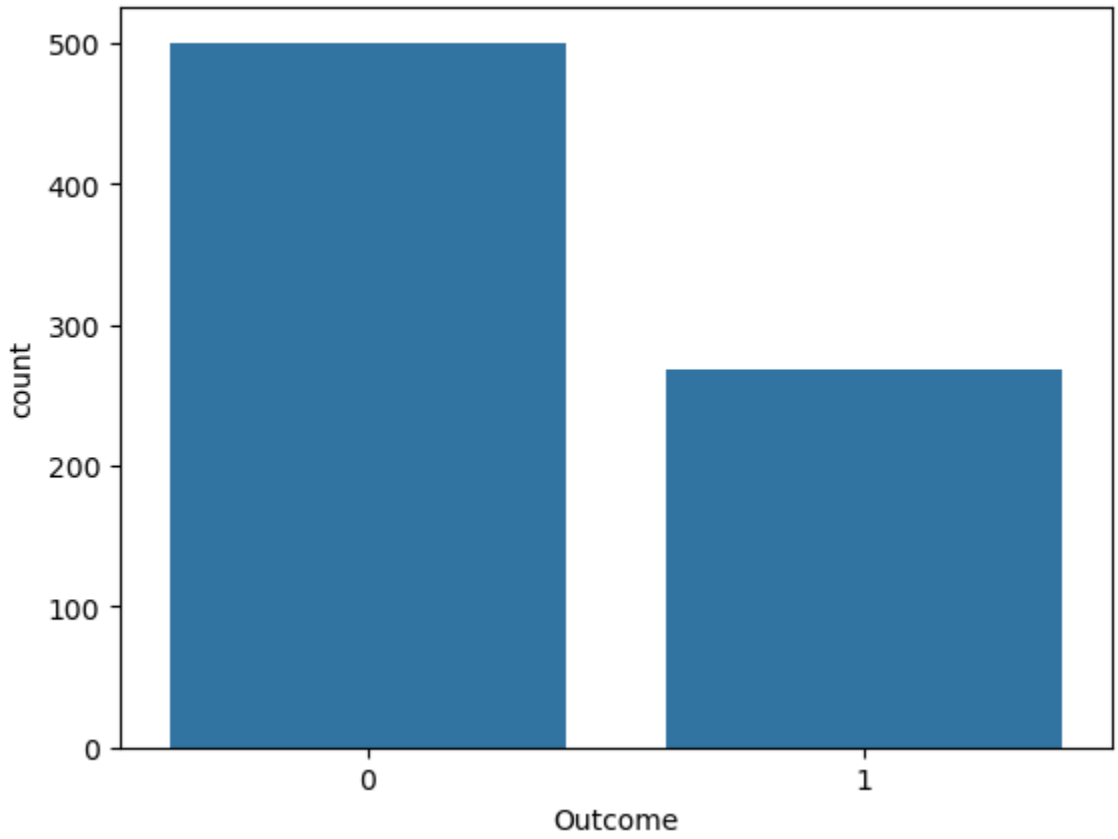
In [4]:

```
x = df.drop('Outcome', axis=1)
y = df['Outcome']
```

In [5]:

```
sns.countplot(x=y);
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



In [6]:

```
y.value_counts()
```

Out[6]:

```
Outcome
0    500
1    268
Name: count, dtype: int64
```

In [7]:

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
x_scaled = scaler.fit_transform(x)
```

In [8]:

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    x_scaled, y, random_state=0, test_size=0.25)
```

In [9]:

```
x.shape
```

Out[9]:

```
(768, 8)
```

In [10]:

```
x_train.shape
```

Out[10]:

```
(576, 8)
```

In [11]:

```
x_test.shape
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Out[11]:
(192, 8)

In [12]:
```
from sklearn.neighbors import KNeighborsClassifier
```

In [13]:
```
knn = KNeighborsClassifier(n_neighbors=5)
```

In [14]:
```
knn.fit(x_train, y_train)
```

Out[14]:

▾ KNeighborsClassifier

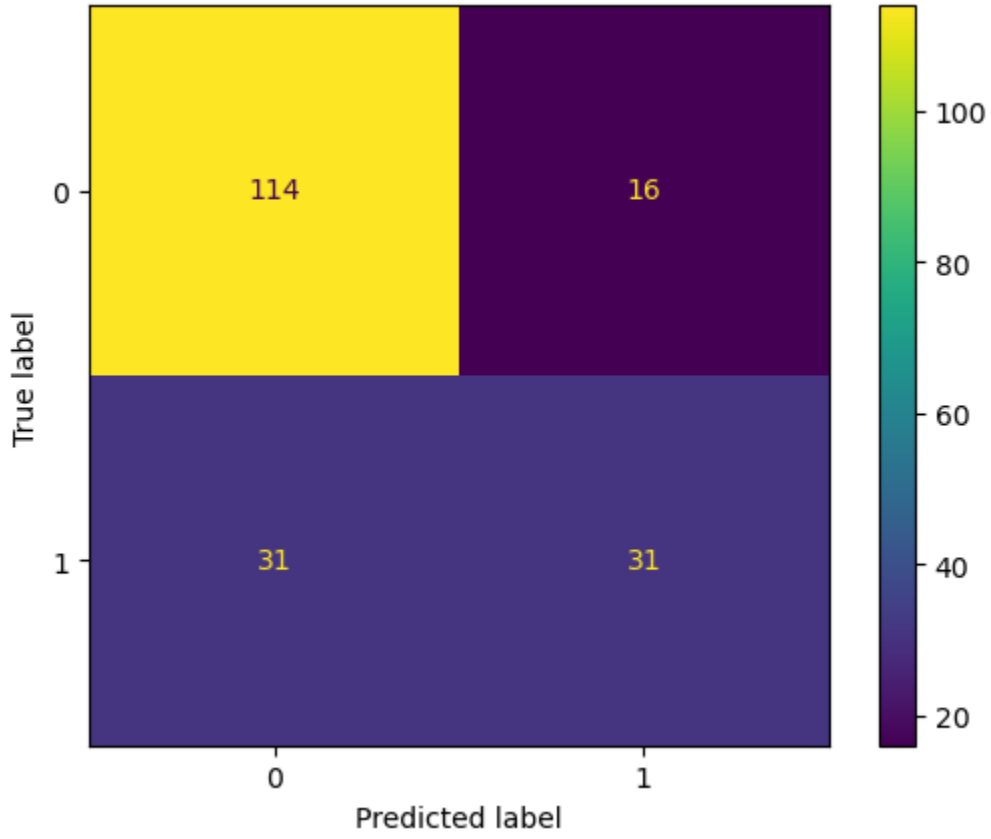
▶ Parameters

In [15]:
```
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay
from sklearn.metrics import classification_report
```

In [16]:
```
y_pred = knn.predict(x_test)
```

In [17]:
```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
```

Out[17]:
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x14d67489a90>



In [18]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.79      0.88      0.83       130
           1       0.66      0.50      0.57        62

    accuracy                           0.76       192
   macro avg       0.72      0.69      0.70       192
weighted avg       0.75      0.76      0.75       192
```

In [19]:

```
import matplotlib.pyplot as plt
import numpy as np
```

In [20]:

```
error = []
for k in range(1, 41):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(x_train, y_train)
    pred = knn.predict(x_test)
    error.append(np.mean(pred != y_test))
```

In [21]:

```
error
```

Out[21]:

```
[np.float64(0.328125),
 np.float64(0.2708333333333333),
 np.float64(0.24479166666666666),
 np.float64(0.2708333333333333),
 np.float64(0.24479166666666666),
 np.float64(0.2552083333333333),
 np.float64(0.234375),
 np.float64(0.24479166666666666),
 np.float64(0.21875),
 np.float64(0.24479166666666666),
 np.float64(0.22916666666666666),
 np.float64(0.22916666666666666),
 np.float64(0.22395833333333334),
 np.float64(0.234375),
 np.float64(0.23958333333333334),
 np.float64(0.21875),
 np.float64(0.21354166666666666),
 np.float64(0.22916666666666666),
 np.float64(0.22916666666666666),
 np.float64(0.22395833333333334),
 np.float64(0.21875),
 np.float64(0.22395833333333334),
 np.float64(0.22395833333333334),
 np.float64(0.22916666666666666),
 np.float64(0.234375),
 np.float64(0.22916666666666666),
 np.float64(0.22395833333333334),
 np.float64(0.21875),
 np.float64(0.22395833333333334),
 np.float64(0.21354166666666666),
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

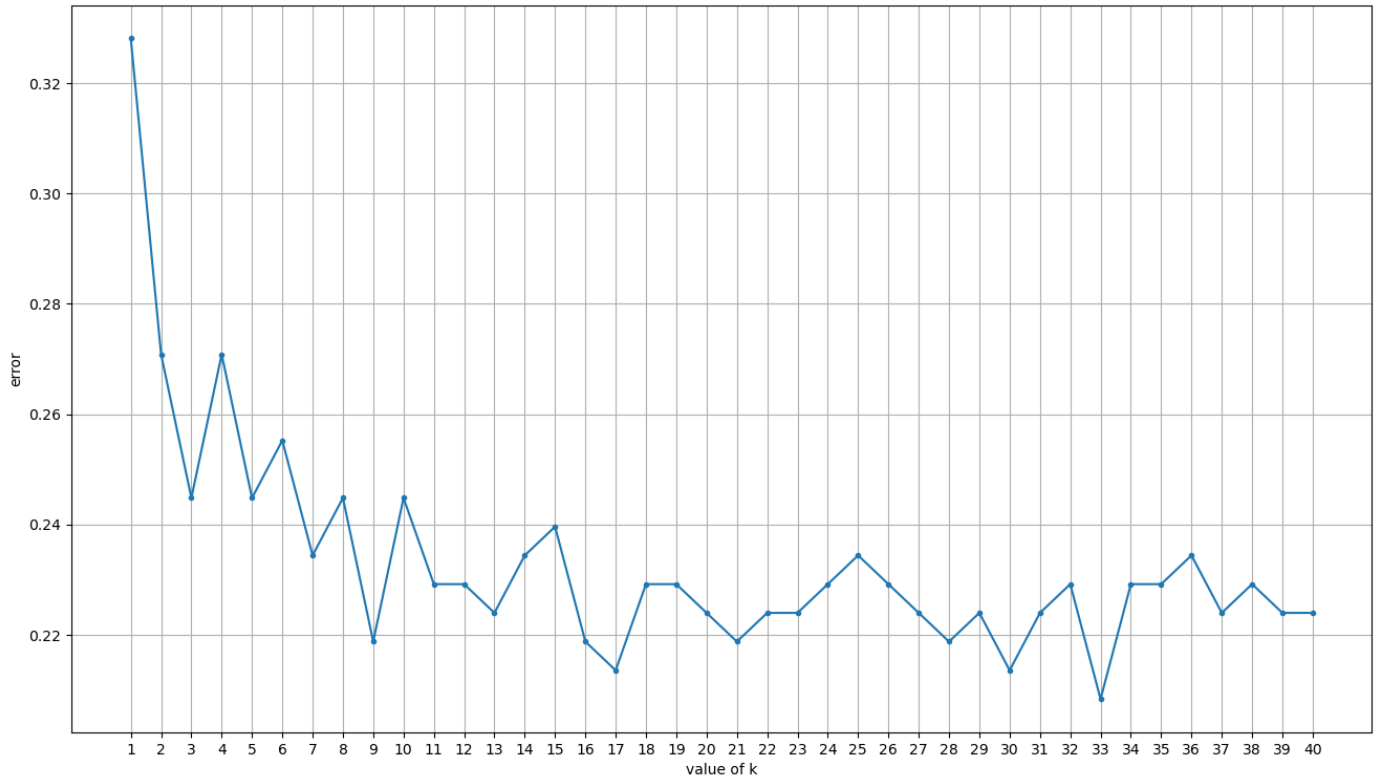
```
 np.float64(0.22395833333333334),
 np.float64(0.22916666666666666),
 np.float64(0.20833333333333334),
 np.float64(0.22916666666666666),
 np.float64(0.22916666666666666),
 np.float64(0.234375),
 np.float64(0.22395833333333334),
 np.float64(0.22916666666666666),
 np.float64(0.22395833333333334),
 np.float64(0.22395833333333334)]
```

In [22]:

```
plt.figure(figsize=(16,9))
plt.xlabel('value of k')
plt.ylabel('error')
plt.grid()
plt.xticks(range(1,41))
plt.plot(range(1,41), error, marker='.')
```

Out[22]:

```
[<matplotlib.lines.Line2D at 0x14d67688690>]
```



In [23]:

```
knn = KNeighborsClassifier(n_neighbors=33)
```

In [24]:

```
knn.fit(x_train, y_train)
```

Out[24]:

▾ KNeighborsClassifier

▶ Parameters

In [25]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
y_pred = knn.predict(x_test)
```

In [26]:

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.79      0.94      0.86       130
           1       0.79      0.48      0.60        62

    accuracy                           0.79       192
   macro avg       0.79      0.71      0.73       192
weighted avg       0.79      0.79      0.78       192
```

In [ ]:

```

```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF